

Spring Boot Starter Dependencies - Detailed Training Notes

1. What Are Starter Dependencies?

Spring Boot **Starter Dependencies** are convenient dependency descriptors that group the libraries normally required for a particular application capability.

Instead of manually identifying and adding many related libraries, developers usually add one starter.

For example, for a standard Spring MVC web/REST application:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

The core idea is:

```
Application Requirement
    |
    v
Choose Spring Boot Starter
    |
    v
Required libraries are added
    |
    v
Spring Boot manages compatible versions
    |
    v
Auto-configuration detects the classpath
    |
    v
Application infrastructure is configured
```

2. Why Do We Need Starters?

Without starters, a developer may need to identify many individual dependencies for:

- Spring Core
- Spring Context
- Spring MVC
- JSON processing
- Logging
- Servlet support
- Embedded server support
- JPA/Hibernate
- Validation
- Security
- Testing

The developer would also need to ensure compatible versions.

With Spring Boot, we normally think in terms of **capabilities**:

Requirement	Typical Starter
REST API / Spring MVC	spring-boot-starter-web
JPA persistence	spring-boot-starter-data-jpa
Validation	spring-boot-starter-validation
Security	spring-boot-starter-security
Monitoring	spring-boot-starter-actuator
Testing	spring-boot-starter-test
Reactive Web	spring-boot-starter-webflux
JDBC	spring-boot-starter-jdbc
Thymeleaf	spring-boot-starter-thymeleaf

3. Starter vs Dependency Management

These concepts work together but solve different problems.

Starter Dependency

Answers:

Which libraries do I normally need for this capability?

Example:

```
spring-boot-starter-web
```

Dependency Management

Answers:

Which compatible versions of those libraries should be used?

For dependencies managed by Spring Boot, we normally omit the version:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Spring Boot's dependency management supplies the compatible managed version.

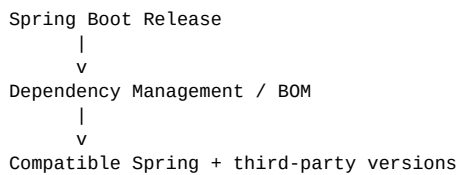
4. Spring Boot Parent and BOM

A Maven project may use spring-boot-starter-parent:

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>YOUR_SPRING_BOOT_VERSION</version>
  <relativePath/>
</parent>
```

Projects that cannot use the Boot parent can import the Spring Boot **BOM (Bill of Materials)** through dependency management.

Conceptually:



Do not copy an arbitrary old version from a tutorial. Use the Spring Boot version selected for your project.

5. spring-boot-starter-web

Used for standard servlet-based Spring MVC web applications and REST APIs.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Typical uses:

- REST APIs
- `@RestController`
- Spring MVC
- JSON request/response handling
- Embedded servlet container

Example:

```
@RestController
@RequestMapping("/employees")
public class EmployeeController {

    @GetMapping
    public String employees() {
        return "Employee List";
    }
}
```

6. Embedded Server and the Web Starter

A normal Spring Boot servlet web application can run with an embedded server.

```
@SpringBootApplication
public class EmployeeApplication {

    public static void main(String[] args) {
        SpringApplication.run(EmployeeApplication.class, args);
    }
}
```

Run:

```
mvn spring-boot:run
```

or:

```
java -jar employee-service.jar
```

For the common executable-JAR model, we do not need to manually install a separate Tomcat server and deploy the application into it.

7. spring-boot-starter-data-jpa

Used for relational persistence with Spring Data JPA.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

It provides the infrastructure needed for areas such as:

- Spring Data JPA
- JPA
- Hibernate ORM integration
- Repository abstraction
- Transaction integration

Example:

```
public interface EmployeeRepository
    extends JpaRepository<Employee, Long> {
}
```

Important: Database Driver Is Separate

The JPA starter does **not** mean PostgreSQL or MySQL is automatically selected.

For PostgreSQL:

```
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <scope>runtime</scope>
</dependency>
```

Think of it as:

```
JPA Starter -> Persistence/ORM infrastructure
JDBC Driver -> Communication with chosen database
Database    -> PostgreSQL/MySQL/etc.
```

8. spring-boot-starter-validation

Used for Bean Validation.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

Example:

```

public class EmployeeRequest {

    @NotBlank
    private String name;

    @Email
    private String email;

    @Min(18)
    private int age;
}

```

Controller:

```

@PostMapping
public Employee create(
    @Valid @RequestBody EmployeeRequest request) {
    return employeeService.create(request);
}

```

9. spring-boot-starter-security

Used for Spring Security integration.

```

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>

```

Common requirements include:

- Authentication
- Authorization
- Security filters
- Password handling
- Method security
- Additional OAuth2/JWT support when corresponding modules are included

Example:

```

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {

        http.authorizeHttpRequests(auth -> auth
            .requestMatchers("/public/**").permitAll()
            .anyRequest().authenticated()
        );

        return http.build();
    }
}

```

10. spring-boot-starter-actuator

Used for production-oriented management and monitoring.

```

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>

```

```
</dependency>
```

Common capabilities:

- Health
- Metrics
- Application information
- Loggers
- Diagnostics
- Observability integrations

Example:

```
management.endpoints.web.exposure.include=health,info
```

Typical endpoint:

```
/actuator/health
```

11. spring-boot-starter-test

Used for Spring Boot testing support.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
```

It provides a convenient testing toolkit around Spring's testing infrastructure and commonly used test libraries.

Example:

```
@SpringBootTest
class EmployeeApplicationTests {

    @Test
    void contextLoads() {
    }

}
```

Exact transitive dependencies can vary between Spring Boot versions.

12. spring-boot-starter-webflux

Used for reactive web applications.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webflux</artifactId>
</dependency>
```

Common concepts:

- Spring WebFlux
- Reactive programming

- Mono
- Flux
- Non-blocking request processing
- WebClient

Example:

```
@GetMapping("/products")
public Flux<Product> products() {
    return productService.findAll();
}
```

Do not select WebFlux simply because it is newer. Choose it when reactive/non-blocking architecture fits the application.

13. spring-boot-starter-jdbc

Used for JDBC-based database access without requiring JPA ORM.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-jdbc</artifactId>
</dependency>
```

Example:

```
@Repository
public class EmployeeDao {

    private final JdbcTemplate jdbcTemplate;

    public EmployeeDao(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }
}
```

14. Other Starter Categories

Depending on the project and Spring Boot ecosystem/version, starters are available for capabilities such as:

- Thymeleaf
- Mail
- Cache
- AMQP
- Batch
- WebSocket
- OAuth2
- Data MongoDB
- Data Redis
- AOP

Always verify the appropriate dependency in the documentation for the Spring Boot version used by the project.

15. What Is a Transitive Dependency?

A transitive dependency is a library that enters our project because a dependency we directly declared requires it.

Example:

```
Our Application
|
v
spring-boot-starter-web
|
+-- Web library
+-- MVC library
+-- JSON support
+-- Logging support
+-- Server-related dependencies
```

We declare the starter, and Maven/Gradle resolves the dependency graph.

16. Inspecting Dependencies

For Maven:

```
mvn dependency:tree
```

This helps answer:

- Which dependency brought this library?
- Are multiple versions involved?
- What was included transitively?
- Was something excluded?
- Why is a particular library on the classpath?

Another useful command is:

```
mvn help:effective-pom
```

It helps inspect inherited and effective Maven configuration.

17. Dependency Scopes

Compile

Default scope. Available during compilation and runtime.

Runtime

Needed at runtime but not normally required to compile application source.

Example:

```
<scope>runtime</scope>
```

Database drivers are often runtime dependencies.

Test

Used only for tests:

```
<scope>test</scope>
```

18. Excluding a Transitive Dependency

Maven supports exclusions.

```
<dependency>
  <groupId>some.group</groupId>
  <artifactId>some-starter</artifactId>

  <exclusions>
    <exclusion>
      <groupId>dependency.group</groupId>
      <artifactId>dependency-name</artifactId>
    </exclusion>
  </exclusions>
</dependency>
```

Use exclusions only after understanding why the dependency exists and what will replace it.

19. Excluding Tomcat and Using Jetty

A common demonstration is replacing the default embedded servlet container.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>

  <exclusions>
    <exclusion>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-tomcat</artifactId>
    </exclusion>
  </exclusions>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-jetty</artifactId>
</dependency>
```

Flow:

```
spring-boot-starter-web
|
X-- spring-boot-starter-tomcat
   excluded

spring-boot-starter-jetty
|
v
Jetty used as embedded servlet container
```

Always verify server compatibility for the Spring Boot generation used by the project.

20. Starters and Auto-Configuration

This relationship is essential.

A starter makes libraries available on the classpath.

Spring Boot auto-configuration examines the classpath and other conditions and can configure appropriate infrastructure.

```
Add Starter
|
```

```

      v
Libraries available on classpath
      |
      v
Auto-configuration conditions evaluated
      |
      v
Spring configures suitable beans
      |
      v
Developer focuses on application logic

```

21. Starter vs Auto-Configuration

Starter

What dependencies should be available?

Auto-Configuration

What infrastructure can Spring configure automatically based on those dependencies and application conditions?

Together:

```

Starter Dependencies
      +
Auto-Configuration
      +
Externalized Configuration
      =
Spring Boot Productivity

```

22. Starter vs @SpringBootApplication

A starter is a **dependency concept**.

@SpringBootApplication is an **annotation used to bootstrap/configure the application**.

Conceptually, @SpringBootApplication combines:

```

@SpringBootConfiguration
@EnableAutoConfiguration
@ComponentScan

```

Therefore:

```

Starter -> Provides libraries
@SpringBootApplication -> Bootstraps/configures Spring Boot application
Auto-configuration -> Configures infrastructure conditionally

```

23. Complete REST API Dependency Example

A typical Spring Boot REST API using PostgreSQL could contain:

```

<dependencies>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
</dependencies>

```

```

</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <scope>runtime</scope>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>

</dependencies>

```

Architecture:

```

REST Controller -> spring-boot-starter-web
Service         -> Spring-managed application code
Repository      -> spring-boot-starter-data-jpa
Database        -> PostgreSQL driver
Validation      -> spring-boot-starter-validation
Monitoring      -> spring-boot-starter-actuator
Tests           -> spring-boot-starter-test

```

24. Spring Initializr and Starters

Spring Initializr lets developers select capabilities such as:

```

Spring Web
Spring Data JPA
PostgreSQL Driver
Validation
Actuator
Security

```

The generated pom.xml or Gradle build contains the appropriate dependencies.

```

Spring Initializr
|
v
Select Capabilities
|
v
Generate Project
|
v
Build File
|
v
Starter Dependencies

```

25. Gradle Example

```

dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-web'
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'org.springframework.boot:spring-boot-starter-validation'

    runtimeOnly 'org.postgresql:postgresql'

    testImplementation 'org.springframework.boot:spring-boot-starter-test'
}

```

```
}
```

The build syntax changes, but the starter concept remains the same.

26. Why Prefer Starters Over Individual Dependencies?

For standard Spring Boot use cases, starters usually provide:

- Less build configuration
- Better readability
- Curated dependency sets
- Integration with Boot dependency management
- Fewer accidental compatibility problems
- Easier project setup

Instead of manually assembling many web libraries, we can usually add:

```
spring-boot-starter-web
```

27. Overriding Managed Versions

It is possible to override a managed dependency version, but do it only for a clear and tested reason.

Arbitrary overrides can cause:

- `NoSuchMethodError`
- `ClassNotFoundException`
- Binary incompatibility
- Unexpected runtime behavior
- Framework integration problems

Best general rule:

Prefer Spring Boot-managed versions unless the project has a specific verified requirement to override them.

28. Common Dependency Problems

`ClassNotFoundException`

Possible causes:

- Missing dependency
- Wrong scope
- Exclusion
- Packaging problem

`NoSuchMethodError`

Often points to incompatible library versions.

Multiple Versions

Different dependency paths may request different versions of a library.

Wrong Starter

Selecting an inappropriate programming model, such as reactive dependencies for an application designed entirely around traditional blocking MVC, can introduce unnecessary complexity.

29. Dependency Troubleshooting Flow

```
Read exception
|
v
Identify class/library
|
v
mvn dependency:tree
|
v
Find dependency path
|
v
Check Boot-managed versions
|
v
Remove unnecessary explicit versions
|
v
Use exclusion only if necessary
|
v
Clean + rebuild + test
```

Useful commands:

```
mvn dependency:tree
mvn clean package
mvn help:effective-pom
```

30. Custom Starter Concept

Organizations can build reusable internal starters.

Example:

```
company-spring-boot-starter
|
+-- Standard logging
+-- Security conventions
+-- Observability
+-- Common exception handling
+-- Shared internal integrations
```

Applications can then depend on the organization's starter to standardize common infrastructure.

This is an advanced topic and should be designed carefully to avoid hidden behavior and excessive coupling.

31. Best Practices

1. Add only the starters your application needs.

2. Prefer Spring Boot-managed dependency versions.
3. Use `mvn dependency:tree` to understand the classpath.
4. Do not treat starters as magic; understand their purpose.
5. Keep test dependencies in test scope.
6. Use runtime scope appropriately for runtime-only libraries.
7. Be cautious with exclusions.
8. Avoid unnecessary manual version overrides.
9. Keep the Spring Boot version on a supported line appropriate for the project.
10. Avoid mixing arbitrary Spring/Hibernate/Jackson/server versions without checking compatibility.
11. Choose MVC vs WebFlux based on architecture, not trend.
12. Remember that a database driver is separate from JPA.
13. Review dependency changes during framework upgrades.

32. Classroom Demo 1 - Spring Web

Step 1

Create a minimal Spring Boot project.

Step 2

Add:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Step 3

Create:

```
@RestController
public class DemoController {

    @GetMapping("/hello")
    public String hello() {
        return "Hello Spring Boot";
    }
}
```

Step 4

Run the application.

Explain how the application starts as a web application with embedded-server infrastructure.

Step 5

Run:

```
mvn dependency:tree
```

Show students how one starter brings multiple related dependencies.

33. Classroom Demo 2 - JPA

Add:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

Create:

```
public interface EmployeeRepository
    extends JpaRepository<Employee, Long> {
}
```

Then add PostgreSQL separately:

```
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <scope>runtime</scope>
</dependency>
```

Explain:

```
JPA Starter
|
v
Persistence libraries
|
v
Auto-configuration
|
v
Repository infrastructure

PostgreSQL Driver
|
v
Database connectivity
```

34. Classroom Demo 3 - Tomcat to Jetty

1. Start with `spring-boot-starter-web`.
2. Run the application and observe the default embedded servlet container.
3. Exclude `spring-boot-starter-tomcat`.
4. Add `spring-boot-starter-jetty`.
5. Refresh Maven.
6. Run again.
7. Compare startup logs.

This demonstrates:

- Starter dependencies
- Transitive dependencies
- Exclusions
- Embedded servers

- Auto-configuration

35. Interview Questions and Answers

Q1. What is a Spring Boot starter?

A starter is a convenient dependency descriptor providing a curated dependency set for a particular Spring Boot capability.

Q2. Why are starters useful?

They reduce manual dependency declarations and work with Spring Boot dependency management to simplify compatible library selection.

Q3. What does `spring-boot-starter-web` provide?

It is the standard starter for servlet-based Spring MVC web/REST applications and related infrastructure.

Q4. What does `spring-boot-starter-data-jpa` provide?

It provides dependencies and integration for JPA persistence with Spring Data JPA.

Q5. Does JPA starter include PostgreSQL automatically?

No. Add the appropriate database driver separately.

Q6. What is a transitive dependency?

A dependency brought into the project indirectly through another dependency.

Q7. How do you inspect Maven dependencies?

```
mvn dependency:tree
```

Q8. What is the difference between a starter and auto-configuration?

A starter supplies dependencies. Auto-configuration conditionally configures Spring infrastructure based on the classpath, properties, beans, and environment.

Q9. What is a BOM?

BOM means Bill of Materials. It centralizes compatible dependency versions.

Q10. Can we exclude a dependency from a starter?

Yes. Maven exclusions or equivalent Gradle mechanisms can be used.

Q11. How can Tomcat be replaced with Jetty?

Exclude `spring-boot-starter-tomcat` and add `spring-boot-starter-jetty`, while checking compatibility with the Spring Boot version.

Q12. Why don't we normally specify versions for Boot-managed dependencies?

Spring Boot's dependency management already provides compatible managed versions.

Q13. What is spring-boot-starter-test?

It provides a convenient dependency set and Spring support for testing Spring Boot applications.

Q14. Should we override managed dependency versions?

Only when there is a specific, verified reason and compatibility has been tested.

Q15. Is a starter the same as @SpringBootApplication?

No. A starter is a build dependency concept; @SpringBootApplication is an application configuration/bootstrap annotation.

36. Common Student Confusions

Starter != Annotation

Starter -> Dependencies
@SpringBootApplication -> Application bootstrap/configuration

Starter != Auto-Configuration

Starter -> Adds libraries
Auto-Configuration -> Configures infrastructure conditionally

JPA Starter != Database

JPA Starter -> ORM/persistence integration
Driver -> Database communication
Database -> PostgreSQL/MySQL/etc.

37. Easy Analogy

Think of a starter as a **ready-made toolkit**.

Without a toolkit:

Choose screwdriver
Choose wrench
Choose pliers
Choose measuring tools
Check compatibility

With a toolkit:

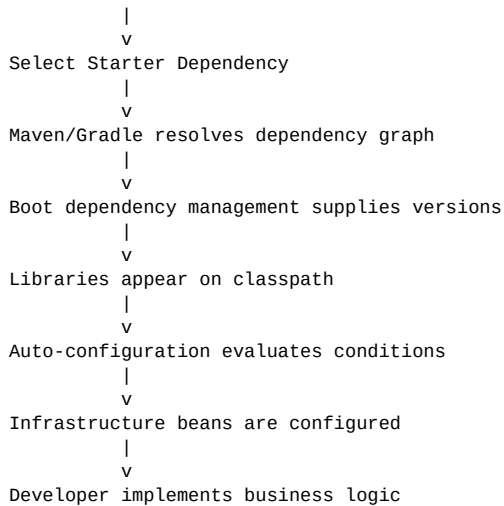
Choose the toolkit for the job
|
v
Common required tools are grouped together

Similarly:

Choose application capability
|
v
Add appropriate Spring Boot starter

38. Complete Starter Flow

Developer identifies requirement



39. Key Takeaways

- Starters simplify dependency setup.
- They group dependencies by application capability.
- Spring Boot dependency management supplies compatible versions for managed dependencies.
- Starters can bring transitive dependencies.
- `spring-boot-starter-web` is commonly used for MVC REST APIs.
- `spring-boot-starter-data-jpa` supports JPA persistence.
- Database drivers are separate dependencies.
- Validation, Security, Actuator, WebFlux, JDBC and Test have dedicated starters.
- Starters and auto-configuration work together but are different concepts.
- Use `mvn dependency:tree` to inspect dependencies.
- Use exclusions carefully.
- Prefer Boot-managed versions unless an override is truly necessary.
- Add only dependencies that the application requires.

40. Quick Revision

```
<!-- Web -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>

<!-- JPA -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<!-- Validation -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>

<!-- Security -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
```

```
</dependency>

<!-- Actuator -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<!-- Test -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
```

Inspect the dependency tree:

```
mvn dependency:tree
```

Final Concept

Spring Boot Starter Dependencies
= Ready-to-use dependency sets
+ Boot-managed compatible versions
+ Integration with auto-configuration
= Less setup + cleaner build files + faster development

Starter dependencies allow developers to describe what capability an application needs instead of manually assembling every supporting library.